

Laboratory report of Nanjing University of Information Science and Technology

Experiment Title Distributed Object Storage & Fault Tolerance Testing

Data 2026/6/11 Class AI 1

Name Weijie Xu ID 202383910029

一、Experimental Purpose

(1) Understanding the Basic Concept of Object Storage

The first purpose of this experiment is to understand the basic concept of object storage. Unlike traditional file systems that manage data through hierarchical file paths, object storage organizes data through buckets and objects. A bucket acts as a logical storage container, while each uploaded file is stored as an independent object. Through this experiment, students can clearly distinguish between conventional directory-based file management and S3-compatible object storage models.

(2) Learning S3-Compatible Bucket and Object Operations

This experiment aims to help students become familiar with common operations in S3-compatible object storage systems. By using MinIO, students can create a bucket, upload a file, list stored objects, and manage cloud objects through the MinIO client. These operations are similar to those used in real cloud storage platforms such as Amazon S3 and Alibaba Cloud OSS. Therefore, the experiment provides practical experience with cloud object storage workflows.

(3) Deploying a Distributed MinIO Storage Cluster

Another purpose of this experiment is to learn how to deploy a distributed storage service using Docker Compose. In the experiment, four independent local directories are created to simulate four storage disks. These directories are mounted into the MinIO container and used together as a distributed storage backend. Through this process, students can understand how containerized services and volume mapping are used to build a simple distributed storage environment.

(4) Observing Backend Data Fragmentation

This experiment also helps students observe how an uploaded object is stored at the backend. After uploading a large test file, students need to inspect the storage directories and compare the internal file structures across different simulated disks.

This allows students to see that the uploaded object is not simply stored as one complete file in a single directory. Instead, MinIO stores object metadata and distributed data fragments across multiple storage locations.

(5) Understanding Erasure Coding and Redundancy

A key purpose of this experiment is to understand the role of erasure coding in distributed object storage. MinIO uses erasure coding to split object data into multiple fragments and distribute them across different storage disks. Some fragments store actual data, while others provide redundancy for recovery. This mechanism enables the system to tolerate partial disk failure while still keeping the object accessible.

(6) Verifying Fault Tolerance Under Simulated Disk Failure

The final purpose of this experiment is to verify the fault tolerance capability of distributed object storage. By deleting one of the simulated storage directories, students can simulate a disk failure scenario. After restarting the MinIO cluster, students check whether the uploaded object is still available. If the object can still be listed and accessed, it demonstrates that the distributed storage system can maintain data availability even when part of the storage backend is damaged.

二、Experiment Contents

- In this experiment, a distributed cloud object storage system is simulated by using MinIO and Docker Compose. MinIO provides an S3-compatible object storage service, while Docker Compose is used to define and start the required containers in a simple and repeatable way. Through this environment, students can build a small-scale object storage system without using a real public cloud platform.
- The experiment requires students to create four independent storage directories to simulate four physical storage disks. These directories are mounted into the MinIO container as separate storage paths. By using multiple storage directories, the experiment creates a distributed storage backend where object data can be stored across different simulated disks.
- Students need to write a docker-compose.yml file to define the MinIO server and the MinIO client container. The file includes important configuration items such as image, container name, volume mapping, port mapping, environment variables, startup command, and service dependency. This step helps students understand how container configuration controls the behavior of a distributed storage service.
- After the configuration file is completed, students will launch the MinIO cluster and verify whether the containers are running correctly. The MinIO server provides the object storage service, while the MinIO client is used to interact with the server

through command-line operations. This process allows students to become familiar with basic container deployment and service verification.

- Students will create a cloud storage bucket and upload a large test file into it. The bucket works as the top-level logical container in object storage, and the uploaded file is stored as an object inside the bucket. This step helps students practice common object storage operations, including bucket creation, file upload, and object listing.
- After uploading the test file, students will inspect the backend storage directories to observe how MinIO stores the object internally. The experiment requires students to compare the directory structures of the four simulated disks and identify metadata files, fragmented storage structures, and distributed object placement. This part helps students understand that object storage hides complex backend data distribution from users.
- Finally, students will simulate a disk failure by deleting one of the storage directories and then restarting the MinIO cluster. After the restart, students need to check whether the uploaded object is still accessible. This step verifies the fault tolerance mechanism of distributed object storage and demonstrates how erasure coding and redundancy help maintain data availability after partial storage failure.

≡ Detailed steps of the experiment

1. Checking Docker and Docker Compose Versions

This screenshot verifies that Docker and Docker Compose are available in the local environment before the distributed storage experiment begins. Confirming the tool versions is necessary because MinIO is deployed through containers, and the later steps depend on Docker Compose to create and manage the required services.

```
@qzxwj → /workspaces/cloud-computing-lab-github (main) $ docker --version
Docker version 29.3.0-1, build 5927d80c76b3ce5cf782be818922966e8a0d87a3
@qzxwj → /workspaces/cloud-computing-lab-github (main) $ docker compose version
Docker Compose version v2.40.3
```

2. Creating Storage Directories

This screenshot shows the creation of the minio-lab working directory and four separate data directories. These directories are used as simulated storage disks, which allows the experiment to build a distributed storage backend on a single local machine.

```
• @qzxwj → /workspaces/cloud-computing-lab-github (main) $ mkdir minio-lab
• @qzxwj → /workspaces/cloud-computing-lab-github (main) $ cd minio-lab
• @qzxwj → /workspaces/cloud-computing-lab-github/minio-lab (main) $ mkdir -p data1 data2 data3 data4
• @qzxwj → /workspaces/cloud-computing-lab-github/minio-lab (main) $ ls
data1 data2 data3 data4
```

3. Writing the Docker Compose Configuration

This screenshot records the docker-compose.yml configuration used in the experiment. The file defines the MinIO server and client containers, volume mappings, exposed ports, login credentials, startup command, and service dependency, so it determines how the distributed object storage service is launched.

```
@qzxwj → /workspaces/cloud-computing-lab-github/minio-lab (main) $ nano docker-compose.yml
@qzxwj → /workspaces/cloud-computing-lab-github/minio-lab (main) $ cat docker-compose.yml
services:
  minio:
    image: quay.io/minio/minio
    container_name: cloud-minio
    volumes:
      - ./data1:/data1
      - ./data2:/data2
      - ./data3:/data3
      - ./data4:/data4
    ports:
      - "9000:9000"
      - "9001:9001"
    environment:
      MINIO_ROOT_USER: admin
      MINIO_ROOT_PASSWORD: password123
    command: server /data{1...4} --console-address ":9001"

  mc:
    image: minio/mc
    container_name: cloud-mc
    depends_on:
      - minio
    entrypoint: /bin/sh
    tty: true
```

4. Starting the MinIO Services

This screenshot shows Docker Compose starting the MinIO services and pulling the required images when necessary. The container list confirms that the MinIO server and client containers have been created successfully and are running in the expected environment.

```
@qzxwj → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker compose up -d
[+] Running 16/16
✓ minio Pulled
✓ 34013573f278 Pull complete
✓ b83ce1c86227 Pull complete
✓ f94d28849fa3 Pull complete
✓ 81260b173076 Pull complete
✓ f9c0805c25ee Pull complete
✓ 1008deaf6ec4 Pull complete
✓ 71e9fc939447 Pull complete
✓ c1bc68842c41 Pull complete
✓ 0288b5a0d7e7 Pull complete
✓ mc Pulled
✓ b7c1efd1d2b4 Pull complete
✓ 4a93952ab9ce Pull complete
✓ b268f447c871 Pull complete
✓ f29310fc4a8a Pull complete
✓ 9404e0df6bd3 Pull complete
[+] Running 3/3
✓ Network minio-lab_default Created
✓ Container cloud-minio Started
✓ Container cloud-mc Started
@qzxwj → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
673fa638fc40   minio/mc      "/bin/sh"               34 seconds ago Up 33 seconds
bb52672c2565   quay.io/minio/minio "/usr/bin/docker-ent..." 38 seconds ago Up 34 seconds 0.0.0.0:9000-9001->9000-9001/tcp,
:::9000-9001->9000-9001/tcp
```

5. Configuring the MinIO Client Alias

This screenshot shows the MinIO client alias being configured for the local MinIO server endpoint. By saving the server address and credentials as an alias, later bucket

and object operations can be performed through shorter and clearer mc commands.

```
@qzxwj → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker exec cloud-mc mc alias set myminio http://minio:9000 admin password123
Added `myminio` successfully.
```

6. Creating the Cloud Storage Bucket

This screenshot shows the creation of the cloud-storage bucket under the configured MinIO alias. The bucket acts as the top-level logical container for objects, similar to a bucket in Amazon S3 or a bucket in Alibaba Cloud OSS.

```
@qzxwj → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker exec cloud-mc mc mb myminio/my-cloud-storage
Bucket created successfully `myminio/my-cloud-storage`.
@qzxwj → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker exec cloud-mc mc ls myminio
[2026-06-11 02:31:31 UTC]    0B my-cloud-storage/
```

7. Generating a Large Test File

This screenshot shows the generation of a large test file named bigfile.bin. A large file is useful in this experiment because it makes backend data distribution easier to observe after the object is uploaded into the MinIO bucket.

```
@qzxwj → /workspaces/cloud-computing-lab-github/minio-lab (main) $ dd if=/dev/urandom of=bigfile.bin bs=1M count=20
20+0 records in
20+0 records out
20971520 bytes (21 MB, 20 MiB) copied, 0.0829957 s, 253 MB/s
@qzxwj → /workspaces/cloud-computing-lab-github/minio-lab (main) $ ls -lh bigfile.bin
-rw-rw-rw- 1 codespace codespace 20M Jun 11 02:32 bigfile.bin
```

8. Checking the Local Test File

This screenshot confirms that bigfile.bin exists locally and has the expected file size before upload. This verification prevents later upload or storage observations from being affected by a missing or incorrectly generated test file.

```
@qzxwj → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker cp bigfile.bin cloud-mc:/bigfile.bin
Successfully copied 21MB to cloud-mc:/bigfile.bin
@qzxwj → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker exec cloud-mc ls -lh /bigfile.bin
-rw-rw-rw- 1 1000 1000 20M Jun 11 02:32 /bigfile.bin
```

9. Uploading and Listing the Object

This screenshot shows the test file being uploaded to the cloud-storage bucket and then listed through the MinIO client. The successful listing proves that the object has been accepted by the object storage service and can be managed through S3-compatible operations.

```
@qzxwj → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker exec cloud-mc mc cp /bigfile.bin myminio/my-cloud-storage/
e/
`/bigfile.bin` -> `myminio/my-cloud-storage/bigfile.bin`
```

Total	Transferred	Duration	Speed
20.00 MiB	20.00 MiB	00m00s	26.21 MiB/s

```
@qzxwj → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker exec cloud-mc mc ls myminio/my-cloud-storage/
[2026-06-11 02:34:28 UTC]  20MiB STANDARD bigfile.bin
```

10. Inspecting Backend Storage Disk 1

This screenshot inspects the first simulated storage directory after the upload. The

displayed internal files and folders show that MinIO stores object metadata and data structures on the backend rather than simply placing the original file in one normal directory.

```
@qzxwj → /workspaces/cloud-computing-lab-github/minio-lab (main) $ ls -R ./data1
./data1:
my-cloud-storage

./data1/my-cloud-storage:
bigfile.bin

./data1/my-cloud-storage/bigfile.bin:
1fe23a6c-ea31-42aa-9d03-386508df5770 xl.meta
```

11. Inspecting Backend Storage Disk 2

This screenshot checks the second simulated storage directory and compares its internal layout with the other disks. The presence of related object data confirms that MinIO distributes storage information across multiple backend locations.

```
@qzxwj → /workspaces/cloud-computing-lab-github/minio-lab (main) $ ls -R ./data2
./data2:
my-cloud-storage

./data2/my-cloud-storage:
bigfile.bin

./data2/my-cloud-storage/bigfile.bin:
1fe23a6c-ea31-42aa-9d03-386508df5770 xl.meta
```

12. Inspecting Backend Storage Disk 3

This screenshot shows the third simulated storage directory. By observing that this directory also contains object-related storage structures, the experiment confirms that the uploaded object is placed across the distributed backend rather than stored on a single disk.

```
@qzxwj → /workspaces/cloud-computing-lab-github/minio-lab (main) $ ls -R ./data3
./data3:
my-cloud-storage

./data3/my-cloud-storage:
bigfile.bin

./data3/my-cloud-storage/bigfile.bin:
1fe23a6c-ea31-42aa-9d03-386508df5770 xl.meta
```

13. Inspecting Backend Storage Disk 4

This screenshot examines the fourth simulated storage directory and displays the object-related erasure-code files. These backend structures help demonstrate how MinIO splits object data and stores fragments with redundancy across the available disks.

```
@qzxwj → /workspaces/cloud-computing-lab-github/minio-lab (main) $ ls -R ./data4
./data4:
my-cloud-storage

./data4/my-cloud-storage:
bigfile.bin

./data4/my-cloud-storage/bigfile.bin:
1fe23a6c-ea31-42aa-9d03-386508df5770 xl.meta

./data4/my-cloud-storage/bigfile.bin/1fe23a6c-ea31-42aa-9d03-386508df5770:
part.1 part.2
```

14. Simulating Disk Failure

This screenshot shows the MinIO cluster being stopped and one storage directory being removed. Deleting a simulated disk directory creates a controlled failure scenario, which is used to test whether the distributed object storage system can still keep the uploaded object available.

```
@qzxwj → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker compose down
[+] Running 3/3
  ✓ Container cloud-mc          Removed          10.2s
  ✓ Container cloud-minio       Removed          0.2s
  ✓ Network minio-lab_default    Removed          0.1s
@qzxwj → /workspaces/cloud-computing-lab-github/minio-lab (main) $ rm -rf data3
@qzxwj → /workspaces/cloud-computing-lab-github/minio-lab (main) $ mkdir data3
```

15. Restarting the Cluster After Failure

This screenshot shows the MinIO services being restarted after the simulated disk failure. The container status confirms that the service can be brought back online even though part of the backend storage has been removed.

```
@qzxwj → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker compose up -d
[+] Running 3/3
  ✓ Network minio-lab_default    Created          0.1s
  ✓ Container cloud-minio        Started          0.4s
  ✓ Container cloud-mc           Started          0.7s
@qzxwj → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker ps
CONTAINER ID   IMAGE          NAMES      COMMAND                  CREATED        STATUS        PORTS
f4955be0847a   minio/mc      cloud-mc   "/bin/sh"               9 seconds ago Up 9 seconds
24c7d06386ad   quay.io/minio/minio   cloud-minio "/usr/bin/docker-ent..." 10 seconds ago Up 9 seconds   0.0.0.0:9000-9001->9000-9001/tcp, [::]:9000-9001->9000-9001/tcp
```

16. Verifying Object Availability After Failure

This screenshot verifies that the uploaded object can still be listed after one simulated storage disk has been deleted. This result demonstrates the fault tolerance provided by MinIO erasure coding and redundancy.

```
@qzxwj → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker exec cloud-mc mc alias set myminio http://minio:9000 admin:password123
Added 'myminio' successfully.
@qzxwj → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker exec cloud-mc mc ls myminio/my-cloud-storage/
[2026-06-11 02:34:28 UTC] 20MiB STANDARD bigfile.bin
```

17. Checking Git Status

This screenshot checks the Git working tree after the experiment files have been created. It helps identify which generated files, configuration files, and storage directories are present before deciding what should be tracked in version control.

```
@qzxwj → /workspaces/cloud-computing-lab-github/minio-lab (main) $ git init
Initialized empty Git repository in /workspaces/cloud-computing-lab-github/minio-lab/.git/
@qzxwj → /workspaces/cloud-computing-lab-github/minio-lab (main) $ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    bigfile.bin
    data1/
    data2/
    data3/
    data4/
    docker-compose.yml

nothing added to commit but untracked files present (use "git add" to track)
```

18. Committing Experiment Files

This screenshot shows the experiment files being committed to Git with a meaningful message. Recording the Docker Compose configuration and related experiment files makes the deployment process reproducible and preserves the completed lab work.

```
@qzxcwj → /workspaces/cloud-computing-lab-github/minio-lab (main) $ git add docker-compose.yml
@qzxcwj → /workspaces/cloud-computing-lab-github/minio-lab (main) $ git commit -m "Complete MinIO distributed object storage lab"
[main (root-commit) fca86fa] Complete MinIO distributed object storage lab
1 file changed, 24 insertions(+)
create mode 100644 docker-compose.yml
```

19. Preparing Git Ignore Rules

This screenshot shows a .gitignore file being created and checked. The purpose is to ignore generated storage data while keeping important configuration files, so the repository remains clean and does not store unnecessary backend data fragments.

```
@qzxcwj → /workspaces/cloud-computing-lab-github/minio-lab (main) $ nano .gitignore
@qzxcwj → /workspaces/cloud-computing-lab-github/minio-lab (main) $ cat .gitignore
data1/
data2/
data3/
data4/
bigfile.bin
```

20. Final Commit After Cleanup

This screenshot shows the final Git commit after adding the ignore rules. The commit records the cleanup step and finalizes the experiment repository with reproducible configuration while excluding generated storage data.

```
@qzxcwj → /workspaces/cloud-computing-lab-github/minio-lab (main) $ git add docker-compose.yml .gitignore
@qzxcwj → /workspaces/cloud-computing-lab-github/minio-lab (main) $ git commit -m "Add MinIO distributed object storage experiment"
[main 4b846cd] Add MinIO distributed object storage experiment
1 file changed, 5 insertions(+)
create mode 100644 .gitignore
```

四、Reflection

1. Problems Encountered and Proposed Solutions

(1) During the environment setup stage, one possible problem was the incorrect creation of the working directory or storage directories. For example, if the directory name was written with extra spaces or if the terminal was not located in the correct path, the later docker-compose.yml file and the four data directories might be created in different locations. This would cause Docker Compose to fail to mount the expected storage paths. To solve this problem, the working directory should be checked carefully before each major step. Commands such as pwd and ls can be used to confirm the current location and verify whether data1, data2, data3, and data4 have been created correctly.

(2) Another problem was related to the formatting and configuration of the docker-compose.yml file. Since YAML files are sensitive to indentation, a small

formatting mistake may prevent the MinIO and MinIO Client containers from starting successfully. Incorrect volume mapping, port mapping, environment variables, or startup commands may also lead to container errors. The proposed solution is to check the file structure carefully and make sure that spaces are used instead of tabs. If the cluster fails to start, `docker compose config`, `docker ps -a`, and `docker logs cloud-minio` can be used to locate configuration or startup problems.

(3) During bucket creation and file upload, the MinIO Client might fail to connect to the MinIO server if the alias was not configured correctly or if the server was not fully started. In addition, the upload command may fail if the test file was not copied into the cloud-mc container before uploading. To solve this problem, the running status of the containers should first be checked with `docker ps`. Then, the alias should be configured with the correct endpoint, username, and password. Before uploading, the file size should be verified on the host machine, and the copied file should also be checked inside the client container to ensure that the upload source exists.

(4) When observing backend storage and testing fault tolerance, the internal storage structure of MinIO may be difficult to understand. The uploaded object does not appear as one complete normal file in a single directory. Instead, MinIO stores object metadata and distributed fragments across multiple storage directories. After deleting one storage directory, it may seem that the file should be lost, but the object can still be accessed because MinIO uses erasure coding and redundancy. The solution is to compare the structures of all four data directories, record the metadata and fragment distribution, then restart the cluster and verify the object again through the MinIO Client. If `bigfile.bin` can still be listed after one directory is deleted, it proves that the distributed object storage system has fault tolerance.

2. Explain in 3–5 sentences why data is still available after deleting one storage directory.

After the file was uploaded to MinIO, it was not stored as a single complete file in only one storage directory. Instead, MinIO used erasure coding to split the object into multiple data and parity fragments, which were distributed across different storage directories. When one storage directory was deleted, the remaining directories still contained enough fragments to reconstruct the original object. Therefore, the uploaded file could still be listed and accessed after the simulated disk failure. This result shows that distributed object storage can maintain data availability through redundancy and fault tolerance mechanisms.

附录 程序命令代码

附录 1: docker_command

```
# -----  
# Step 0: Check Docker environment  
# -----  
docker --version  
docker compose version  
  
# -----  
# Step 1: Create a dedicated lab directory  
# -----  
mkdir minio-lab  
cd minio-lab  
  
# -----  
# Step 2: Create four storage directories  
# These directories simulate four independent storage disks.  
# -----  
mkdir -p data1 data2 data3 data4  
  
# Verify the directories  
ls  
  
# -----  
# Step 3: Create docker-compose.yml  
# -----  
cat > docker-compose.yml <<'EOF'  
services:  
  minio:  
    image: quay.io/minio/minio  
    container_name: cloud-minio  
    volumes:  
      - ./data1:/data1  
      - ./data2:/data2
```

```
- ./data3:/data3
- ./data4:/data4

ports:
- "9000:9000"
- "9001:9001"

environment:
  MINIO_ROOT_USER: admin
  MINIO_ROOT_PASSWORD: password123
  command: server /data{1...4} --console-address ":9001"

mc:
  image: minio/mc
  container_name: cloud-mc
  depends_on:
    - minio
  entrypoint: /bin/sh
  tty: true
EOF

# Verify the docker-compose.yml file
cat docker-compose.yml

# Optional: validate Docker Compose configuration
docker compose config

# -----
# Step 4: Launch the distributed MinIO cluster
# -----

docker compose up -d

# Verify running containers
docker ps

# Optional: check MinIO logs if needed
docker logs cloud-minio
```

```
# -----  
# Step 5: Configure MinIO Client alias  
# -----  
docker exec cloud-mc mc alias set myminio http://minio:9000 admin password123  
  
# -----  
# Step 6: Create a bucket  
# -----  
docker exec cloud-mc mc mb myminio/my-cloud-storage  
  
# Verify the bucket  
docker exec cloud-mc mc ls myminio  
  
# -----  
# Step 7: Create a 20 MB test file  
# -----  
dd if=/dev/urandom of=bigfile.bin bs=1M count=20  
  
# Verify the file size  
ls -lh bigfile.bin  
  
# -----  
# Step 8: Copy the test file into the MinIO Client container  
# -----  
docker cp bigfile.bin cloud-mc:/bigfile.bin  
# Verify the file inside the container  
docker exec cloud-mc ls -lh /bigfile.bin  
  
# -----  
# Step 9: Upload the file to the bucket  
# -----  
docker exec cloud-mc mc cp /bigfile.bin myminio/my-cloud-storage/  
# -----
```

```
# Step 10: Verify object upload
# -----
docker exec cloud-mc mc ls myminio/my-cloud-storage/

# -----
# Step 11: Observe backend storage fragmentation
# Compare the storage structure across four simulated disks.
# -----
ls -R ./data1
ls -R ./data2
ls -R ./data3
ls -R ./data4

# -----
# Step 12: Simulate disk failure
# Stop the cluster first.
# -----
docker compose down

# Delete one storage directory to simulate disk failure
rm -rf data3
# Recreate an empty storage directory
mkdir data3
# Verify the directory exists again
ls

# -----
# Step 13: Restart the distributed cluster
# -----
docker compose up -d
# Verify running containers
docker ps

# -----
# Step 14: Reconfigure MinIO Client alias if needed
# This step is optional but recommended after restart.
```

```
# -----  
docker exec cloud-mc mc alias set myminio http://minio:9000 admin password123  
  
# -----  
# Step 15: Verify fault tolerance  
# The uploaded object should still be accessible.  
# -----  
  
docker exec cloud-mc mc ls myminio/my-cloud-storage/  
  
# Optional: download the object from MinIO to further verify accessibility  
docker exec cloud-mc mc cp myminio/my-cloud-storage/bigfile.bin  
/downloaded-bigfile.bin  
  
# Verify the downloaded file inside the container  
docker exec cloud-mc ls -lh /downloaded-bigfile.bin  
  
# -----  
# Step 16: Save changes using Git  
# It is recommended not to commit generated data directories or big files.  
# -----  
cat > .gitignore <<'EOF'  
data1/  
data2/  
data3/  
data4/  
bigfile.bin  
EOF  
  
git init  
git add docker-compose.yml .gitignore  
git commit -m "Add MinIO distributed object storage experiment"
```